General Sir John Kotelawala Defence University

Faculty of Computing



Essentials of Artificial Intelligence

FINAL REPORT

Project Title	AI-Powered Dynamic Public Transport Fleet Optimizer (Route 100)		
Group No.	**Index No.**	**Name**	**Degree**
40	D/BSE/25/0012	M Uvaram	Software Engineering
	D/DBA/25/0016	TTC Jayasinghe	Data Science & Business Analytics
	D/BIT/24/0039	SNMRDS Arambepola	Information Technology
	D/BIT/24/0056	JGJB Gamage	Information Technology

Table of Contents

1. Abstract 4
2. Introduction 5
3. Problem Background 6
4. Problem Statement 7
5. Objectives 7
5.1 General Objective 7
5.2 Specific Objectives 7
6. Selected AI Techniques 8
6.1 Multi-Layer Perceptron Regression 8
6.2 Huber Loss 8
6.3 Adam Optimizer and Early Stopping 8
6.4 Data Preprocessing 8
6.5 Rule-Based Dispatch Recommendation 9
7. System Design 9
7.1 Dataset and Input Features 9
7.2 Model Architecture 11
7.3 Training and Data Leakage Prevention 11
7.4 Dispatch Recommendation logic 11
8. Implementation Details 12
8.1 Technology stack 12
8.2 Backend Implementaion 12
8.3 Frontend dashboard 12
8.4 Team contributions 13
9. Testing and Results 14
9.1 Model performance 14
9.2 Functional Testing 14
9.3 Interpretation of Results 17
10. Limitations and Discussion 17
10.1 Dependence on Synthetic Data 17

10.2 Limited Predictive Accuracy 18
10.3 Simplified Dispatch Logic 18
10.4 Limited Operational Scope 18
10.5 Discussion and Future Improvements 18
11. Ethical and Social Considerations 19
11.1 Human Oversight 19
11.2 Transparency and Uncertainty 19
11.3 Fairness and Accessibility 19
11.4 Privacy and Data Protection 20
11.5 Workforce and Accountability 20
11.6 Environmental Considerations 20
12. Conclusion 21
13. References 22

1. Abstract

One common problem related to public transit services is that passengers are not equally distributed, the waiting time is long, and the buses get crowded, resulting in bus bunching. This is particularly true in the case of major city streets, such as Galle Road in Sri Lanka. Timetable-based public transit dispatching strategies have little flexibility to accommodate variations in traffic demands, which can be caused by weather, schooling, holidays, and other events.

The proposed project presents an AI powered Dynamic Public Transport Fleet Optimizer for SLTB Route 100 between Panadura and Pettah in Sri Lanka. The solution includes a system with MLP regression models built on PyTorch that makes the prediction of the number of passengers expected at eight strategic stations along the Galle Road corridor. The predicted demand data is processed through rules-based dispatch optimization and the recommendations for the optimal bus frequency, dispatch headway, fleet size, and corridor travel time are provided. A decision support interface developed on FastAPI and Next.js helps to analyze different operating scenarios.

The model has been trained on a total of 800,000 records divided into training and testing sets chronologically in order to eliminate data leakage from the past into the future. The MAE equals 225.8 passengers, RMSE equals 519.3, and the coefficient of determination (R^2) equals 0.33 for the 158,622 held out records. Ten test cases have been successful.

The results demonstrate that all four components such as demand prediction, dispatch recommendation algorithm, back-end service, and dashboard are well-integrated. However, the accuracy of the model in question is limited because the dataset used for the development of the prototype is synthetic and not based on real passenger boarding data. Consequently, the developed prototype should be viewed as the first decision point in a series of decisions to be made and not as a deployable system but as a tool that supports making decisions.

2. Introduction

Public transport is a vital key in ensuring people can get to work, engage in economic activities and move around in the city as part of everyday living in Sri Lanka. Galle Road is a key transportation link in the city between Panadura and Colombo passing through multiple populated urban centers. Pettah to Panadura service is provided by SLTB Route 100 through the following stops; Moratuwa, Mount Lavinia, Dehiwala, Wellawatta, Bambalapitiya and Kollupitiya.

There is a wide variation in passenger demand on this line throughout the day. The morning peak periods tend to create higher demand to the central Colombo levels and the evening peak periods create higher demand from the opposite direction. There may be other variations due to school time, weather, public holiday, pay day, or special events. Therefore, the timetables may not always accurately reflect passenger demand, or the current traffic circumstances.

Bus bunching is one of the major operational problems that are created due to uneven demand and service delays, which is the occurrence of two or more buses at a stop at close intervals, followed by a period of time with no buses at the stop. This may lead to overcrowded trains, lack of efficient use of the train fleet, and irregular passenger waiting times.

Aiming to overcome these issues, this project will use AI to forecast passenger demand and then transform the forecast into dynamic dispatch recommendations. It's an integrated system built with machine learning, an operational layer of rules, backend API services, and a web-based dashboard, designed to help depot managers assess service needs in various scenarios.

This report presents our project's problem background, problem statement, objectives, selected AI techniques, system design, implementation, testing results, limitations, ethical considerations, and recommendations for future development.

3. Problem Background

One of the typical operational issues in public transportation systems is bus bunching. This can occur if a bus has been delayed and finds more people waiting at the next stop. If the bus can board more passengers, it will get even further back in line. In the meantime, the subsequent bus that runs may have fewer passengers and run faster, catching up with the late bus. This process can result in varying service frequencies, overcrowding and an ineffective allocation of transport service.

Some previous works have focused on the problem of predicting and controlling bus bunching using smart card data, robust dynamic control in the presence of uncertainty in bus demand, and reinforcement learning for bus speed control. The following studies provide a good idea for the significance of having operational data and prediction methods to use in conjunction with one another for public transportation services.

The conventional mode of dispatching based on timetables has its drawbacks since passenger demand is dependent on various factors amongst which are:

- **Time-related factors:** Hour of the day, day of the week, and seasonal variations.
- **Location-related factors:** Bus stop, travel direction, and the distribution of passenger demand along the corridor.
- **Weather conditions:** Rainfall and other weather-related disruptions.
- **Calendar conditions:** Public holidays, bank holidays, school holidays, and Poya days.
- **Special events:** Paydays, public gatherings, rallies, and other events that may increase passenger demand.

When these variations are not adequately considered, transportation operators may experience overcrowding during peak periods, underutilization during off-peak periods, and inconsistent service intervals.

An adaptive decision-support system could help address these issues by estimating passenger demand at individual stops and providing recommendations that reflect changing operational conditions.

4. Problem Statement

With varying operating conditions, SLTB Route 100 should have the capacity to respond to the demands for passengers and help with the dispatch decisions. Timetable planning may not be sufficient in addressing the varying demands resulting from peak travel hours, weather, holiday periods, special events, and school schedules.

Without an adaptive and demand-responsive decision support system, the service headway can greatly change, buses can be overloaded, there could be inefficiency in the number of buses, and there will be increased waiting time.

Therefore, the objective of the current project is to develop an artificial intelligence-based prototype that can predict the demand for passengers at selected bus stops of Route 100 and convert such predictions into actionable dispatch decisions. The proposed model is expected to help the depot managers make informed decisions on operations through an interactive dashboard.

The system cannot auto-dispatch buses and auto-control them but provides decision support to humans.

5. Objectives

5.1 General Objective

To develop an AI-powered decision-support system that forecasts passenger demand and generates dynamic bus dispatch recommendations for SLTB Route 100.

5.2 Specific Objectives

1. To construct a hybrid dataset incorporating transport-related, temporal, calendar, weather, and special-event information.
2. To develop and evaluate a deep learning regression model for predicting passenger demand at eight selected stops along the Route 100 corridor.
3. To implement a rule-based dispatch recommendation mechanism that converts predicted passenger demand into suggested fleet requirements and dispatch headways.
4. To provide an interactive dashboard that enables depot managers to evaluate different operating conditions and review demand forecasts.
5. To assess the predictive performance of the model using appropriate regression evaluation metrics.
6. To verify the functional correctness of the forecasting pipeline, dispatch logic, API endpoints, and scenario simulation features.
7. To identify the system's limitations and recommend improvements required for future real-world implementation.

6. Selected AI Techniques

6.1 Multi-Layer Perceptron Regression

The choice of Multi-Layer Perceptron (MLP) as a model of research of interrelation between passenger demand and several input parameters was made. Passengers numbers may depend on the complex of interactions between time, movement direction, stop, weather and calendar parameters. Input parameters may have a nonlinear relationship, which can be learned by the MLP.

The MLP was built using PyTorch library and had 3 hidden layers of 128, 64 and 32 neurons correspondingly. MLP had Batch Normalization and ReLU activation function, and Dropout (0.2 rate) was applied to the first two hidden layers to prevent overfitting.

Output layer gave continuous predictions of passenger demand.

6.2 Huber Loss

Huber loss, also known as Smooth L1 loss in PyTorch, was used as the model's training objective. Passenger demand datasets may contain unusually high values during peak periods or special events. Huber loss combines the characteristics of squared-error and absolute-error losses, reducing the influence of large individual errors compared with conventional squared-error loss.

This approach was selected to support model training when the target values contain substantial variations.

6.3 Adam Optimizer and Early Stopping

The Adam optimization algorithm was used to update the network parameters, with an initial learning rate of 0.001. Early stopping was applied to monitor validation performance and terminate training when further training no longer produced sufficient improvement.

These techniques were intended to support stable optimization and reduce unnecessary training.

6.4 Data Preprocessing

Data preprocessing was performed using Scikit-learn. A OneHotEncoder was used to represent categorical variables, including travel direction, stop name, and weather condition. A MinMaxScaler was applied to normalize numerical and binary input features.

The encoder and scaler were fitted exclusively on the training partition and subsequently applied to the remaining data. This procedure was designed to reduce information leakage from the evaluation data into the training process.

6.5 Rule-Based Dispatch Recommendation

A transparent, rule-based dispatch module was developed to convert predicted passenger demand into operational recommendations. Rather than implementing a search algorithm such as A*, the system uses predefined mathematical rules to estimate fleet requirements and dispatch headways.

This approach was selected because its calculations are straightforward to interpret and can be reviewed by depot personnel. However, the module represents a preliminary dispatch heuristic rather than a comprehensive optimization algorithm.

7. System Design

The proposed system follows a six-stage processing pipeline, beginning with dataset preparation and ending with the presentation of dispatch recommendations through the dashboard.

Stage	Component	Main Function
1	Data / ETL	Builds the hybrid Route 100 dataset from transit, calendar, weather and event information
2	Pre-processing	Cleans data, creates features, encodes and scales them
3	MLP model	Predicts passenger demand per stop
4	FastAPI backend	Serves predictions and dispatch recommendations, and enriches the calendar
5	Dispatch logic (dispatch.py)	Converts demand into buses, headway and corridor ETA
6	Next.js dashboard	Allows depot staff set conditions and view results

7.1 Dataset and Input Features

Because actual passenger boarding records were unavailable for the project, a hybrid dataset containing approximately 800,000 records was constructed using stratified sampling across travel directions and hours.

The dataset represents eight major stops along the Galle Road corridor:

1. Panadura
2. Moratuwa
3. Mount Lavinia
4. Dehiwala
5. Wellawatta
6. Bambalapitiya
7. Kollupitiya
8. Pettah

The dataset was designed to represent selected characteristics of Colombo commuting patterns. These included increased inbound demand during the morning peak, increased outbound demand during the evening peak, a 1.40× school-rush multiplier during selected hours, Sri Lankan public and bank holidays identified using a holiday library, a modelled monsoon rainfall effect, and an assumed special-event demand increase of approximately 5%.

These characteristics represent modelling assumptions rather than measurements of actual passenger behavior.

Following encoding and preprocessing, the model used 23 input features.

Feature Category	Variables
Categorical	Travel direction, stop name, weather condition
Temporal	Hour, day of the week, month
Calendar	Weekend indicator, public holiday, school holiday, payday
Operational	Morning/evening rush, school rush, special-event override
Prediction target	Passenger count per stop

The categorical and numerical variables were transformed into a numerical feature representation suitable for the neural network.

7.2 Model Architecture

The neural network was composed of an input layer, three hidden layers, and a one output neuron.

The architecture was structured as follows:

23 inputs → Dense (128) → Batch Normalization → ReLU → Dropout (0.2) → Dense (64) → Batch Normalization → ReLU → Dropout (0.2) → Dense (32) → ReLU → Output (1).

The output neuron represents the predicted number of passengers for a given set of stops and operating scenario.

7.3 Training and Data Leakage Prevention

The dataset was split chronologically, with the oldest 80% of records became the train set and the newest 20% -the test set. We did this to evaluate the model on records that were not present during training.

In particular, to prevent leakage between train and test sets, we only fit preprocessing components on the train set. The resulting model weights, preprocessing pipeline and evaluation metrics were exported for the backend API to use. The exported artifacts were as follows:

- demand_predictor.pth — contains weights of the trained pytorch model
- preprocessor.pkl — contains preprocessing pipeline fitted on the train set
- metrics.json — contains model evaluation metrics.

While chronological splitting helps to prevent temporal leakage, the use of synthetic data means that evaluation only reflects the model's ability to generalize to the synthetic dataset, not to actual SLTB operations.

7.4 Dispatch Recommendation logic

The dispatch layer has three simple steps:

1. **Find the bottleneck -** peak = max(predicted_demand)
2. **Size the fleet -** buses = max(1, ceil(peak / 80)), since every bus carries 80 passengers.
3. **Set the headway -** headway = clamp(round(60 / buses), 3, 30) minutes.

For example, if the peak demand is 400 passengers, then we need ceil(400 / 80) = 5 buses, and 60 / 5 = 12 minutes between them. The clamp function ensures that the recommendation is not lower than 3 minutes and higher than 30 minutes.

The corridor ETA is estimated as the baseline free-flowing multiplied by a peak congestion factor of the rush hour, plus a dwell delay of 0.05 × the predicted passengers at each stop. This makes busier stops therefore lengthen the journey, as they would in reality.

8. Implementation Details

8.1 Technology stack

Component	Technology
Data processing	Python, Pandas, PyArrow
Machine learning	PyTorch, Scikit-Learn
Backend API	FastAPI, Uvicorn, Pydantic
Frontend	Next.js, Tailwind CSS, TypeScript
Weather data	Open-Meteo API

8.2 Backend Implementaion

The FastAPI backend connects the frontend dashboard to the trained machine learning model and dispatch recommendation module.

It is initialized with the trained model and preprocessing pipeline for the chosen demand prediction, and provides endpoints to the service health check (/health and /), demand prediction (/predict), dispatch recommendations (/dispatch), and model evaluation metrics (/metrics).

A calendar-enrichment component derives relevant temporal information from the date provided by the user, including the day of the week, weekend indicator, Sri Lankan public holiday information, school-rush indicator, and peak-period categorization.

8.3 Frontend dashboard

The frontend is implemented in Next.js, TypeScript, and Tailwind CSS. It provides a depot-planning interface through which users can configure the operating conditions and view the recommendations generated.

The dashboard allows users to select:

- Traffic direction, date and hour
- Weather; clear and rainy
- Day category; working day, Poya day, and government holidays
- School-vacation and school-rush settings.
- Special-events

Weather information can be pre-populated using an Open-Meteo API for Colombo, subject to data availability.

After the user generates an optimization scenario, the dashboard will displays:

1. Suggested dispatch headway and buses per hour.
2. Predicted peak demand and estimated corridor travel time.
3. A visual representation of the 08 stops, highlighting the one with the highest predicted demand.
4. A comparison bar chart of the predicted demand for the selected stops

The interface will be geared towards aiding the depot personnel to comprehend demand trends and compare alternate scenarios visualize the alternatives and support decision-making.

8.4 Team contributions

Index Number	Name	Contribution
D/BSE/25/0012	M Uvaram	AI/ML development: MLP implementation, model training and evaluation, and model integration
D/DBA/25/0016	TTC Jayasinghe	Dataset development, feature engineering, and data/model evaluation support
D/BIT/24/0039	SNMRDS Arambepola	Dispatch logic and backend API
D/BIT/24/0056	JGJB Gamage	Frontend dashboard and backend API support

9. Testing and Results

The system was assessed with quantitative model evaluation and functional testing. The former evaluated the performance of the trained MLP, whereas the latter tested the dashboard, scenario simulator, backend API, and dispatch logic.

9.1 Model performance

The model was assessed using 158,622 held-out records. The following metrics were recorded in metrics.json.

Metric	Result	Interpretation
MAE	225.8 passengers (about 226)	Average absolute difference between predicted and target passenger counts.
RMSE	519.3	Error metric that assigns greater weight to larger prediction errors.
R^2	0.33	Approximately 33% of the target variance is explained by the model.

The MAE metric indicates that the predictions differed from the target values by an average of approximately 226 passengers in absolute value. The RMSE was considerably higher than the MAE, suggesting that some observations had substantially larger errors than others.

An R^2 value of 0.33 indicates fairly poor explanatory performance. Consequently, the model should be treated as a preliminary forecasting baseline rather than a reliable predictor for actual dispatch decisions.

These metrics must also be interpreted in the context of the synthetic dataset. They do not constitute evidence that the passenger-demand predictions would have been accurate for real-world Route 100 services.

9.2 Functional Testing

Functional testing was carried out across three areas: the depot-manager dashboard, the scenario simulator, and the backend API.

1. **Depot Manager View – Dynamic Dispatch and Headway Recommendation**

Test ID	Scenario	Test input	Expected result	Status
TC-DV-01	Weekday morning peak (rush hour)	Weekday, 07:30, Inbound_to_Pettah, Clear; click	Heavy congestion predicted at CBD hubs (Pettah, Kollupitiya); peak	Pass

Test ID	Scenario	Test input	Expected result	Status
		Generate Optimization	demand above 80 pax; tight headway (6 min or less) with 10 or more buses; ETA reflects rush-hour delay	
TC-DV-02	Midday off-peak commute	Weekday, 12:00, Inbound_to_Pettah, Clear	Moderate demand; peak stop load within single-bus capacity; relaxed headway (10–15 min); free-flow ETA	Pass
TC-DV-03	Late night / boundary demand	Weekday, 02:00, Outbound_to_Panadura	Low demand (under 15 pax per stop); headway clamps at the 30-minute ceiling to prevent idling	Pass

Table 1: Depot Manager View

2. Scenario Simulator and Colombo Overrides

Test ID	Scenario	Test input	Expected result	Status
TC-SC-01	Monsoon rain shock	Hour 08:00, Inbound_to_Pettah; weather set to Rain	Stop counts and dwell times increase; ETA lengthens; headway tightens to absorb crowds	Pass
TC-SC-02	School rush surge	School-term weekday, 06:30 (or 13:30)	is_school_rush = 1; 1.40× multiplier applied; more buses allocated before office commuters arrive	Pass

Test ID	Scenario	Test input	Expected result	Status
TC-SC-03	Poya day / public holiday override	Hour 08:00; holiday mode set to Poya Day	is_public_holiday = 1; morning CBD rush drops noticeably; headway eases to 15–20 min	Pass
TC-SC-04	Special event / rally surge	Event override active (about 5% special-event surge)	Stop volume rises by more than 1.8×; bottleneck stop shows a high-demand badge; fleet size scales up	Pass

Table 2: Scenario Simulator and Colombo Overrides

3. Backend API and Boundary Edge Cases

(unit and integration checks in backend/test_verification.py)

Test ID	Scenario	Test input	Expected result	Status
TC-API-01	Service health check	GET /health or GET /	HTTP 200 with service metadata (PyTorch MLP, 8 stops registered, model input dimension 23)	Pass
TC-API-02	Headway floor clamping	Simulated massive crowd spike (peak above 2,000 pax)	headway = clamp(round(60 / buses), 3, 30); never drops below the 3-minute safety floor	Pass
TC-API-03	Calendar auto-enrichment	Raw ISO date 2026-10-07 sent to POST /dispatch	API detects Wednesday, sets is_weekend = 0, checks the Sri Lankan holiday calendar and assigns the correct peak classification	Pass

Table 3: Backend API and Boundary Edge Cases

Overall functional verification result: All ten test cases passed.

During demonstrations, the dashboard responded to changes in the selected conditions and updated its forecasts and dispatch recommendations. The morning-peak scenario produced shorter recommended headways than the midday off-peak scenario, while the weather, school-rush, and event controls influenced the outputs according to the configured model and rules.

These results demonstrate successful functional integration. However, scenario-based tests establish that the system responds according to its implemented logic; they do not independently establish the accuracy of the forecasts or the effectiveness of the recommendations under real operating conditions.

9.3 Interpretation of Results

It was shown that the model could effectively incorporate the data pre-processing, demand forecasting, dispatch calculation, backend systems, and dashboard as the frontend application.

Chronological split is more appropriate to assess the sequence of temporally ordered data than random split. But since model-based assumptions were made while observations, it was impossible to use metrics derived from these observations for generalization for real passenger demand.

The model's prediction accuracy can be viewed as the key weakness of the model, as demonstrated by its low R^2 value of 0.33 and MAE of 225.8 passengers. Also, there are simplifications made in the dispatch model, such as maximum demand on the corridor and the capacity of the bus.

Consequently, the current implementation represents one example of an AI-based prototype but requires additional empirical testing.

10. Limitations and Discussion

10.1 Dependence on Synthetic Data

The only limitation is the lack of genuine passenger boarding data. Hybrid data were synthesized by making certain assumptions about commuting behavior, schedule of school days, weather conditions, holidays, and special occasions.

The assumptions are made for developing and verifying the prototype, yet they may not hold true for the passenger behavior. Thus, the model will demonstrate different behavior when applied to the actual transportation data.

Future work should include acquisition of the anonymized boarding counts and other operational data from SLTB, followed by further training and verification.

10.2 Limited Predictive Accuracy

This model achieved an MAE of 225.8 passengers, and R^2 of 0.33. This implies that the predictions have high uncertainty levels and therefore are still not ready for practical application.

Possible future work might include hyperparameter tuning, improvement of feature construction process, benchmarking against simpler models, and implementation of other algorithms. All these measures can be beneficial for model generalization and need real-world data for implementation.

10.3 Simplified Dispatch Logic

In the current dispatch algorithm, it is assumed that the fleet demands are set up by the maximum number of possible passengers at any stop, and each bus has a nominal capacity of 80 passengers.

It is extremely simple to use but cannot be utilized for modeling the passenger turnover between stops, occupancy of vehicles, presence of vehicles on the route, location of vehicles, traffic situation, driver availability, scheduling.

Also, the system lacks automatic tracking of vehicles' locations and detecting vehicle bunching.

Further versions of this prototype will contain GPS location data, real occupancy of vehicles, boarding/alighting pattern, etc.

10.4 Limited Operational Scope

The current prototype supports 1 bus route and 8 bus stops which were chosen. Also, weather and other special effects are modeled with the use of a few programmable scenarios.

It is not continuously updating data on passengers and vehicles but only gives suggestions depending on the selected conditions. So, its operational character is not verified yet.

10.5 Discussion and Future Improvements

Despite these limitations, the project demonstrates how the machine learning and rule-based decision support can be integrated into a public transportation planning application.

The architecture provides a foundation for future development through improved data collection, model validation, and more advanced dispatch recommendations. Potential extensions include:

- Obtaining real passenger boarding counts and vehicle GPS data.
- Comparing the MLP with baseline regression models and other suitable machine learning algorithms.

- Evaluating performance separately across stops, directions, peak periods, weather conditions, and holidays.
- Incorporating current fleet availability and vehicle positions into dispatch recommendations.
- Extending the system to additional routes, such as Routes 120, 138, and 177, subject to data availability.
- Exploring multi-route fleet allocation and more advanced operational optimization.

These improvements would help establish whether the proposed approach can deliver measurable benefits in passenger waiting times, fleet utilization, and service reliability.

11. Ethical and Social Considerations

11.1 Human Oversight

The system is designed to support, rather than replace, depot managers and transport operations personnel. Dispatch recommendations should remain subject to human review, particularly because the model has not been validated using actual passenger-demand data.

11.2 Transparency and Uncertainty

Forecasts may appear precise even when their underlying accuracy is limited. The dashboard should therefore clearly identify predictions as estimates and communicate the limitations of the model.

The current evaluation results should not be used to imply that the system can reliably forecast real-world demand without further validation.

11.3 Fairness and Accessibility

More effective allocation of transport capacity could potentially improve access to public transportation and reduce overcrowding. However, a forecasting model may inadequately represent quieter stops, less frequent travel periods, or groups whose travel patterns are not reflected in the training data.

Future evaluation should examine performance across different stops, travel directions, and operating periods to identify potential disparities in service recommendations.

11.4 Privacy and Data Protection

The current prototype uses simulated passenger counts and does not require personally identifiable passenger information.

If future versions incorporate smart-card records, mobile application data, or other passenger-level information, appropriate data minimization, anonymization, access controls, and compliance with applicable Sri Lankan data protection requirements will be necessary.

11.5 Workforce and Accountability

The system is intended to assist transport personnel in evaluating operating conditions. Responsibility for dispatch decisions should remain with authorized SLTB personnel, supported by clear procedures for reviewing recommendations, overriding unsuitable outputs, and investigating operational issues.

11.6 Environmental Considerations

Improved fleet allocation may create opportunities to reduce unnecessary vehicle idling, inefficient operations, and fuel consumption. More reliable public transport could also support greater use of shared transportation.

However, these benefits have not been measured in the current prototype. Any environmental or service-efficiency improvements would need to be evaluated using actual operational data.

12. Conclusion

This project developed an AI-powered decision-support prototype for dynamic public transportation planning on SLTB Route 100 between Panadura and Pettah. The system integrates a hybrid data-processing pipeline, a PyTorch-based MLP demand-forecasting model, a rule-based dispatch recommendation module, a FastAPI backend, and a Next.js dashboard.

The prototype was designed to represent variations in passenger demand associated with time, travel direction, stop location, weather, school schedules, public holidays, and special events. It provides predicted passenger demand, recommended dispatch headways, estimated fleet requirements, and corridor travel-time estimates through an interactive interface.

The model was evaluated on 158,622 held-out records, achieving an MAE of 225.8 passengers, an RMSE of 519.3 passengers, and an R^2 of 0.33. All ten functional and API verification test cases passed, demonstrating that the implemented components operate together according to the defined requirements.

Nevertheless, the results must be interpreted cautiously. The model's predictive accuracy is limited, and the use of synthetic data prevents the evaluation from establishing performance on actual passenger demand. In addition, the dispatch calculations rely on simplified assumptions and do not incorporate real-time vehicle positions or existing fleet conditions.

Overall, the project demonstrates the technical feasibility of integrating AI-based demand forecasting with an interactive public transportation decision-support system. Future development should focus on acquiring real operational data, improving model accuracy, incorporating live vehicle information, and evaluating dispatch recommendations against measurable service outcomes. With these improvements, the proposed approach could provide a foundation for more adaptive and data-informed public transportation planning.

the semester we built a data pipeline, trained a PyTorch MLP on a hybrid dataset of about 800,000 rows, wrapped it in a FastAPI service with calendar enrichment, added a transparent headway and ETA calculator, and presented it all in a Next.js dashboard that responds immediately to rain, school rush, holidays and special events. All ten test cases passed, confirming the system behaves sensibly from input to recommendation.

The honest part of the story is the model's accuracy: an MAE of 225.8 passengers, an RMSE of 519.3 and an R^2 of 0.33 on 158,622 held-out records. That is a useful baseline, not a deployable result, and it mostly reflects the limits of synthetic data. With real passenger data, continued tuning and richer dispatch logic, we believe this approach can grow into a genuine decision-support tool for SLTB and other busy corridors. For us, the project was a hands-on lesson in how AI, software engineering and real-world operations have to work together.

13. References

1. Andres, M., & Nair, R. (2017). A predictive-control framework to address bus bunching. *Transportation Research Part B: Methodological*, *104*, 123–148. https://doi.org/10.1016/j.trb.2017.06.013
2. Comi, A., Sassano, M., & Alessio, V. (2022). Monitoring and controlling real-time bus services: A reinforcement learning procedure for eliminating bus bunching. *24th Euro Working Group on Transportation Meeting*, *62*, 302–309. https://doi.org/10.1016/j.trpro.2022.02.038
3. Daganzo, C. F. (2009). A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons. *Transportation Research Part B: Methodological*, *43*(10), 913–921. https://doi.org/10.1016/j.trb.2009.04.002
4. Li, S., Liu, R., Yang, L., & Gao, Z. (2019). Robust dynamic bus controls considering delay disturbances and passenger demand uncertainty. *Transportation Research Part B: Methodological*, *123*, 88–109. https://doi.org/10.1016/j.trb.2019.03.019
5. Liu, D., Xiao, F., Luo, J., & Yang, F. (2023). Deep reinforcement learning-based holding control for bus bunching under stochastic travel time and demand. *Sustainability*, *15*(14), 10947. https://doi.org/10.3390/su151410947
6. Yu, H., Chen, D., Wu, Z., Ma, X., & Wang, Y. (2016). Headway-based bus bunching prediction using transit smart card data. *Transportation Research Part C: Emerging Technologies*, *72*, 45–59. https://doi.org/10.1016/j.trc.2016.09.007